Scoring Rubric — AI Readiness Training Program

Section 1: Overview

Total Score Formula

```
Total Score = (Phase1_Score × 0.15) + (Phase2_Score × 0.20) + (Phase3_Score × 0.20) + (Phase4_Score × 0.25) + (Phase5_Score × 0.20)
```

Each phase score is calculated as:

```
Phase_Score = (Tests_Passed / Total_Tests) × 100
```

Phase Weight Table

Phase	Name	Total Test Cases	Weight	Max Contribution
Phase 1	Full Stack CRUD	20	15%	15 points
Phase 2	RAG Application	20	20%	20 points
Phase 3	Context Engineering	20	20%	20 points
Phase 4	MCP + Chat Interface	25	25%	25 points
Phase 5	Multi-Agent LangGraph	25	20%	20 points
Total		**110**	**100%**	**100 points**

Pass/Fail Criteria Per Phase

Phase	Tests Required to Pass	Minimum Pass Score
Phase 1	≥14 of 20 tests passed	70%
Phase 2	≥14 of 20 tests passed	70%
Phase 3	≥14 of 20 tests passed	70%
Phase 4	≥18 of 25 tests passed	70% (rounded)
Phase 5	≥18 of 25 tests passed	70% (rounded)

> A phase is **cleared** when the pass threshold is met. All 5 phases must be attempted, even if earlier phases were not cleared.

Section 2: Performance Tier Definitions

Tier	Symbol	Criteria	Next Steps
Elite Performer	🏆	All 5 phases cleared (≥70% in each)	POCs are presented to Account Delivery Heads to demonstrate competencies and enhance opportunities for project allocations
Emerging Contributor	⭐	Exactly 4 phases cleared	Only a select group of shortlisted NGAs will be invited to present the POC to the Account Delivery Heads (ADHs)
Foundational Builder	🌱	3 or fewer phases cleared	Will receive additional mentorship and be assigned targeted training programs to enhance their skills and meet the expected standards

---

Section 3: Phase-by-Phase Test Case Breakdown

Phase 1 — Full Stack CRUD (20 Test Cases)

Category	Count	What It Tests	Points Each
Unit Tests	8	Individual service/model/utility methods (business logic, validation, calculations)	1 point
API Integration Tests	8	HTTP endpoints via TestClient/HttpClient/MockMvc — request/response, status codes, auth	1 point
Database Tests	4	CRUD persistence, constraints, cascade rules, data integrity	1 point
Total	**20**		**20 points**

Category Descriptions:

- **Unit Tests:** Test individual functions in isolation. No HTTP calls, no database. Mock dependencies. Examples: validate email format, calculate EMI, check status transition rules.
- **API Integration Tests:** Use the test client to send HTTP requests to your running API. Verify response status codes, response body structure, authentication enforcement.
- **Database Tests:** Verify data is actually persisted correctly in SQLite. Test constraint violations, cascade deletes, relationship integrity.

Pass Criteria:

- Unit: Function returns expected output for given input; raises correct exception for invalid input
- API: Returns correct HTTP status code (200/201/400/401/404) with correct response schema
- Database: Record exists/doesn't exist in DB with correct field values

---

Phase 2 — RAG Application (20 Test Cases)

Category	Count	What It Tests	Points Each
Ingestion Tests	4	Document loading, chunking, embedding, ChromaDB persistence	1 point
Retrieval Tests	6	Semantic search accuracy, top-k results, relevance scoring	1 point
Generation Tests	6	LLM answer quality, grounding, format, hallucination prevention	1 point
Observability Tests	4	LangSmith trace creation, OTel span export, log field completeness	1 point
Total	**20**		**20 points**

Category Descriptions:

- **Ingestion Tests:** Verify the RAG pipeline can load the user manual, split it into correct-sized chunks, generate embeddings, and store them in ChromaDB persistently.
- **Retrieval Tests:** Given a query, verify that semantically relevant chunks are returned. Test with in-scope questions (should retrieve), out-of-scope questions (low similarity score), and edge cases (empty query).
- **Generation Tests:** Verify the LLM produces answers grounded in retrieved context. Use keyword matching and similarity checks. Test that answers do NOT make up information not in the manual.
- **Observability Tests:** Verify that LangSmith traces are created per query, OpenTelemetry spans are exported, and structured log fields are present.

AI Quality Evaluation for Generation Tests:

- Grounding check: Answer must contain information from retrieved chunks (keyword overlap ≥ 60%)
- Format check: Answer must be a non-empty string
- Relevance check: Answer must address the question topic
- Hallucination check: For out-of-scope questions, answer must indicate the information is not available

---

Phase 3 — Context Engineering & Tool Integration (20 Test Cases)

Category	Count	What It Tests	Points Each
Tool Definition Tests	4	Tool schema validation, registration, descriptions	1 point
Tool Execution Tests	6	API call accuracy, response parsing, error handling	1 point
Context Management Tests	4	Prompt construction, context injection, length limits	1 point
End-to-End Reasoning Tests	6	Multi-step queries requiring tool chaining, final answer quality	1 point
Total	**20**		**20 points**

Category Descriptions:

- **Tool Definition Tests:** Verify each LangChain `@tool` has a valid name, description, and input schema. Verify all tools are registered in the agent.
- **Tool Execution Tests:** Call each tool with valid and invalid inputs. Verify it returns correctly parsed data from the Phase 1 API. Verify graceful error handling when the API is unavailable.
- **Context Management Tests:** Verify the system prompt template renders correctly with context injection. Verify long API responses are summarized before being sent to the LLM. Verify total context stays within model limits.
- **End-to-End Reasoning Tests:** Send a natural language query that requires ≥2 tool calls. Verify the agent selects the correct tools, executes them in the right order, and produces a coherent final answer.

Phase 4 — MCP + Chat Interface (25 Test Cases)

Category	Count	What It Tests	Points Each
MCP Server Tests	8	Tool registration, schema, execution, error handling	1 point
Chat Interface Tests	6	UI loading, message flow, session persistence	1 point
LangChain-MCP Integration Tests	7	Tool discovery, invocation, response routing, multi-turn	1 point
Observability Tests	4	LangSmith traces, OTel spans, session IDs in logs	1 point
Total	**25**		**25 points**

Category Descriptions:

- **MCP Server Tests:** Start the MCP server and verify all tools are discoverable via the MCP protocol. Test each tool with valid inputs and verify correct response. Test invalid inputs return proper MCP error responses.
- **Chat Interface Tests:** Load the Streamlit app and verify it renders. Send a message and verify a response appears. Verify conversation history is shown. Verify session ID is assigned.
- **LangChain-MCP Integration Tests:** Verify LangChain can discover MCP tools, invoke them in response to natural language queries, and route responses back to the user. Test multi-turn conversations where context is maintained.
- **Observability Tests:** Verify every MCP tool invocation creates a LangSmith trace. Verify the chat session ID appears in traces. Verify OTel spans are created for MCP calls.

Phase 5 — Multi-Agent LangGraph (25 Test Cases)

Category	Count	What It Tests	Points Each
State Schema Tests	4	TypedDict validation, initialization, mutation	1 point
Individual Agent Tests	8	Each agent's tool use, output format, state updates	1 point
Supervisor Routing Tests	6	Conditional edge logic, agent handoffs, error paths	1 point

Category	Count	What It Tests	Points Each
End-to-End Workflow Tests	7	Full graph execution, final answer quality, LangSmith tracing	1 point
Total	**25**		**25 points**

Category Descriptions:

- **State Schema Tests:** Verify the `TypedDict` state schema is correctly defined. Verify state initializes with correct defaults. Verify agents can read and write state without KeyErrors.
- **Individual Agent Tests:** Test each agent node in isolation by providing mock state input. Verify the agent's output populates the correct state fields. Verify the agent uses its assigned tools.
- **Supervisor Routing Tests:** Test the supervisor's routing logic. Verify it routes to the correct next agent based on state conditions. Test error paths (e.g., if data collection fails, graph should terminate gracefully).
- **End-to-End Workflow Tests:** Execute the full graph from START to END with realistic input data. Verify the final state contains the expected output. Verify all agents are traced in LangSmith.

Section 4: Test Execution Instructions

Python (pytest)

```
# Install test dependencies
pip install pytest pytest-asyncio httpx pytest-cov

# Run all tests for a phase
pytest tests/phase1/ -v

# Run with coverage report
pytest tests/phase1/ -v --cov=app --cov-report=html

# Run specific test category
pytest tests/phase1/ -v -k "unit"

# Generate JUnit XML report for submission
pytest tests/phase1/ --junitxml=results/phase1-results.xml
```

.Net C# (xUnit)

```
# Run all tests
dotnet test

# Run with detailed output
dotnet test --logger "console;verbosity=detailed"

# Run specific test class
```

```
dotnet test --filter "ClassName=Phase1UnitTests"

# Generate test report
dotnet test --logger "trx;LogFileName=phase1-results.trx"
```

Java (JUnit 5 with Maven)

```
# Run all tests
mvn test

# Run specific test class
mvn test -Dtest=Phase1UnitTests

# Generate Surefire report
mvn surefire-report:report

# Run with verbose output
mvn test -Dsurefire.useFile=false
```

Submitting Results

1. Run the full test suite for your completed phase
2. Screenshot or export the test results showing pass/fail count
3. Share the results XML/TRX file with your mentor
4. Mentor verifies results by running the same test suite against your submitted code

---

Section 5: AI Quality Evaluation (Phases 2–5)

For phases involving LLM responses (Phases 2–5), some test cases evaluate **AI output quality**, not just code correctness. These use an **LLM-as-Judge** pattern.

Evaluation Dimensions

Dimension	Description	Minimum Score
Faithfulness	Does the answer only use information from the retrieved context?	≥ 0.7
Answer Relevance	Does the answer address what was asked?	≥ 0.7
Context Precision	Are the retrieved chunks actually relevant to the question?	≥ 0.6
Context Recall	Does the retrieved context contain the information needed to answer?	≥ 0.6

LLM-as-Judge Prompt Template

```
JUDGE_PROMPT = """
You are evaluating an AI assistant's answer.

Question: {question}
Retrieved Context: {context}
Answer: {answer}

Score the answer on these dimensions (0.0 to 1.0):
1. Faithfulness: Does the answer only use information from the context?
2. Relevance: Does the answer address the question?

Return a JSON object: {{"faithfulness": 0.0, "relevance": 0.0, "reasoning":
"..."}}
"""
```

RAGAS Integration (Optional, for advanced associates)

Associates who want to use RAGAS for automated evaluation can install `ragas` and use:

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_precision

result = evaluate(dataset, metrics=[faithfulness, answer_relevancy,
context_precision])
```

Phase 5 Agent Reasoning Evaluation

For Phase 5, evaluators will additionally assess:

- Are the 4 agents performing distinct, specialized tasks?
- Does the supervisor route correctly between agents?
- Is the final output richer than what a single agent could produce?
- Are agent handoffs visible in LangSmith traces?

Section 6: Academic Integrity

Fair Use of AI Tools

- **Permitted:** Using GitHub Copilot for code suggestions, completions, and generation
- **Permitted:** Using LLMs (ChatGPT, Claude, Gemini) for learning concepts, debugging, and understanding errors
- **Permitted:** Referencing official documentation, tutorials, and Stack Overflow
- **Permitted:** Discussing approaches with teammates (but code must be independently written)

Not Permitted

- Copying another associate's implementation verbatim
- Submitting AI-generated code without understanding it (mentors will ask questions during review)
- Sharing test case solutions across associates before evaluation

Plagiarism Checks

Mentors will conduct code walkthroughs during Phase 4 and Phase 5 evaluations. Associates must be able to explain every part of their implementation. Code similarity checks will be run against submissions from the same cohort.

Section 7: Appeals Process

If an associate believes their test results were incorrectly evaluated:

1. Raise the issue with your mentor within **24 hours** of result communication
2. Provide the test output file showing the results
3. Mentor will re-run the test suite against your submitted code within 48 hours
4. If disagreement persists, program coordinator makes the final decision